WEEK 3 · LESSON 18 OF 24 · HANDOUT

Week 3 Project: Ask the Handbook

Study time: **about 120 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=-eRjgj1zww4 — open the lesson online to watch inside the course.

🎓 **Go deeper:** EuroPython 2024: Fundamentals of Retrieval Augmented Generation — https://www.youtube.com/watch?v=vSwK5sBpD2c

🎓 **Go deeper:** PyData Berlin 2023: How to Feed Facts to Large Language Models and Reduce Hallucination — https://www.youtube.com/watch?v=-eRjgj1zww4

Your goal in this lesson: Build, evaluate and improve a complete question-answering assistant over a document collection, and report its quality honestly.

The brief

Your school, church, SACCO, company or community group has rules and information in documents that people keep asking about. Build an **"Ask the Handbook"** assistant that answers from those documents with citations, and prove how good it is.

Start with the practice handbook; then replace it with real documents you are allowed to use (no personal data).

```
import importlib.util, os, subprocess, sys

if importlib.util.find_spec("anthropic") is None:          # install once per Colab session
    subprocess.run([sys.executable, "-m", "pip", "install", "-q", "anthropic"], check=True)

try:
    from google.colab import userdata                     # Colab: key saved in the Secrets
panel (key icon)
    os.environ["ANTHROPIC_API_KEY"] = userdata.get("ANTHROPIC_API_KEY")
except ImportError:
    pass                                                    # not in Colab: set
ANTHROPIC_API_KEY in your environment
except Exception:
    print("Add a secret called ANTHROPIC_API_KEY in Colab's Secrets panel, then run this cell
again.")

import anthropic

client = anthropic.Anthropic()        # reads ANTHROPIC_API_KEY from the environment
MODEL = "claude-haiku-5-5"            # fast and low-cost; check docs.claude.com for current
model names

def ask(prompt, system=None, max_tokens=500):
    extra = {"system": system} if system else {}
    response = client.messages.create(model=MODEL, max_tokens=max_tokens,
                                      messages=[{"role": "user", "content": prompt}], **extra)
    return response.content[0].text
```

```
HANDBOOK = {
    "Opening hours": "The Kigali Tech Hub is open Monday to Friday from 8:00 to 20:00 and on
Saturday "
                     "from 9:00 to 14:00. The hub is closed on Sundays and on public
holidays.",
    "Membership plans": "There are three plans. The Student plan costs 10,000 RWF per month and
needs a "
                        "valid student card. The Standard plan costs 25,000 RWF per month. The
Team plan "
                        "costs 90,000 RWF per month for up to five people.",
    "Joining": "To become a member, fill in the online form, bring your national ID or passport
to the "
               "front desk, and pay the first month. Your access card is ready the same day.",
    "Payment": "Fees are paid by mobile money or bank transfer before the 5th of each month.
Members who "
               "pay late for two months lose access until the balance is cleared.",
    "Cancelling": "You may cancel your membership at any time with 14 days' written notice by
email. "
                  "Fees already paid for the current month are not refunded.",
    "Wi-Fi": "The Wi-Fi network is called KTH-Members. The password is printed on your access
card and "
             "changes every three months. Guests use the separate KTH-Guest network.",
    "Meeting rooms": "Members can book the two meeting rooms through the hub app. Standard and
Team members "
                     "get 4 free hours per month; Student members get 1 free hour. Extra hours
cost 5,000 RWF.",
    "Guests": "Each member may bring up to two guests per day. Guests must sign in at the front
desk "
              "and leave by 18:00.",
    "Printing": "Black-and-white printing costs 50 RWF per page and colour printing 200 RWF per
page. "
                "Students get 100 free black-and-white pages per month.",
    "Events": "The hub runs a free coding workshop every Wednesday at 17:00 and a startup pitch
night "
              "on the last Friday of each month. Members can register in the hub app.",
    "Lockers": "Lockers can be rented for 3,000 RWF per month. Lost locker keys cost 5,000 RWF
to replace.",
    "Code of conduct": "Members must respect others, keep noise low in the quiet zone, and
never share their "
                       "access card. Harassment leads to immediate cancellation of
membership.",
    "Lost and found": "Items left behind are kept at the front desk for 30 days, then
donated.",
    "Accessibility": "The ground floor, meeting rooms and toilets are wheelchair accessible. A
lift serves "
                     "all floors. Contact the front desk to arrange other support.",
}
chunks = [{"id": i, "title": t, "text": f"{t}. {body}"} for i, (t, body) in
enumerate(HANDBOOK.items())]
print(len(chunks), "sections")
```

```
test_questions = [
    ("How much is the student membership?", "Membership plans"),
    ("Can I come to the hub on Sunday?", "Opening hours"),
    ("What is the Wi-Fi password?", "Wi-Fi"),
    ("How do I book a meeting room?", "Meeting rooms"),
    ("If I quit, do I get my money back for this month?", "Cancelling"),
    ("How many friends can I bring with me?", "Guests"),
    ("What does colour printing cost?", "Printing"),
    ("When is the coding workshop?", "Events"),
    ("What documents do I need to sign up?", "Joining"),
    ("What happens if I don't pay my fees on time?", "Payment"),
    ("I left my bag at the hub last week. Is it still there?", "Lost and found"),
    ("Is there a lift for wheelchair users?", "Accessibility"),
]
print(len(test_questions), "labelled questions")
```

Part 1 — Index the documents

```
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import TruncatedSVD
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import Normalizer

texts = [c["text"] for c in chunks]
embedder = make_pipeline(TfidfVectorizer(stop_words="english", sublinear_tf=True),
                         TruncatedSVD(n_components=12, random_state=0),
                         Normalizer())                      # unit length: dot product = cosine
similarity
doc_vectors = embedder.fit_transform(texts)

def embed(text_list):
    return embedder.transform(text_list)

print("each section is now a vector of", doc_vectors.shape[1], "numbers")
```

Part 2 — Retrieval with hybrid search

```
import math, re
from collections import Counter

STOP = {"the", "a", "an", "is", "are", "i", "my", "do", "to", "of", "and", "for", "in", "on",
"at",
        "what", "how", "can", "me", "it", "if", "with", "does", "there", "this", "be", "by",
"or"}
tok = lambda s: [w for w in re.findall(r"[a-z0-9]+", s.lower()) if w not in STOP]
docs_tok = [tok(c["text"]) for c in chunks]
avg_len = sum(map(len, docs_tok)) / len(docs_tok)
df = Counter(w for d in docs_tok for w in set(d))

def bm25(query, k1=1.5, b=0.75):
    out = []
    for d in docs_tok:
        tf = Counter(d)
        out.append(sum(math.log(1 + (len(docs_tok) - df[w] + 0.5) / (df[w] + 0.5))
                       * tf[w] * (k1 + 1) / (tf[w] + k1 * (1 - b + b * len(d) / avg_len))
                       for w in tok(query) if w in tf))
    return np.array(out)

def retrieve(question, k=3):
    fused = Counter()
    for ranking in (np.argsort(doc_vectors @ embed([question])[0])[::-1],
np.argsort(bm25(question))[::-1]):
        for rank, i in enumerate(ranking):
            fused[int(i)] += 1 / (60 + rank + 1)
    return [chunks[i] for i, _ in fused.most_common(k)]

hits = sum(expected in [c["title"] for c in retrieve(q)] for q, expected in test_questions)
print(f"hybrid recall@3: {hits}/{len(test_questions)}")
```

Part 3 — The assistant

```
RAG_SYSTEM = (
    "You are the Kigali Tech Hub assistant. Answer using ONLY the numbered sources and cite
them like [1]. "
    "Keep answers to at most three sentences. If the sources do not contain the answer, reply
exactly: "
    "\"I don't know based on the handbook.\""
)

def ask_handbook(question):
    sources = retrieve(question)
    numbered = "\n".join(f"[{n}] {s['text']}" for n, s in enumerate(sources, 1))
    text = ask(f"<sources>\n{numbered}\n</sources>\n\n<question>{question}</question>",
               system=RAG_SYSTEM, max_tokens=300)
    cited = sorted({int(n) for n in re.findall(r"\[(\d+)\]", text) if 1 <= int(n) <=
len(sources)})
    return {"answer": text, "cited": [sources[n - 1]["title"] for n in cited]}

print(ask_handbook("How much does a locker cost?"))
```

Part 4 — Evaluate

```
eval_set = [
    {"q": "How much is the Team plan?", "facts": ["90,000"], "refuse": False},
    {"q": "When is the startup pitch night?", "facts": ["last Friday"], "refuse": False},
    {"q": "How much does it cost to replace a lost locker key?", "facts": ["5,000"], "refuse":
False},
    {"q": "Until what time can guests stay?", "facts": ["18:00"], "refuse": False},
    {"q": "Does the hub offer free lunch?", "facts": [], "refuse": True},
]

results = []
for case in eval_set:
    out = ask_handbook(case["q"])
    text = out["answer"].lower()
    refused = "i don't know based on the handbook" in text
    results.append({
        "question": case["q"][:40],
        "refusal ok": refused == case["refuse"],
        "facts ok": case["refuse"] or all(f.lower() in text for f in case["facts"]),
        "cited": bool(out["cited"]) or case["refuse"],
    })

import pandas as pd
table = pd.DataFrame(results)
print(table)
print("pass rate:", f"{table[['refusal ok', 'facts ok', 'cited']].all(axis=1).mean():.0%}")
```

Part 5 — Improve one thing at a time

Try at least two changes and re-run Parts 2 and 4 after each: different `k`, smaller chunks, query rewriting, a stricter prompt, a larger model. Record the numbers before and after in a table.

Part 6 — Report (two pages)

1. **Documents:** what, how many, how you chunked them, permission to use them.
2. **Retrieval:** recall@1 and recall@3 for embeddings, BM25 and hybrid.
3. **Answers:** the evaluation table and pass rate; three example answers with their citations.
4. **Improvements:** what you changed and the measured effect.
5. **Risks:** outdated documents, questions in other languages, private information in documents, over-trust by users.
6. **Next steps:** how documents will be kept up to date, and who reviews the assistant's answers.

Marking guide (self-check)

Area	Excellent work
Retrieval	Hybrid search with recall@k measured
Grounding	Answers cite sources; refusals work
Evaluation	Objective checks with a pass rate
Iteration	At least two measured improvements
Responsibility	Permission, privacy and freshness of documents addressed

Remember this: A trustworthy "ask the documents" assistant is built on measured retrieval, grounded answers with citations, honest refusals and an evaluation you can repeat.

Name: _____ Date: _____

Activity

Task 1

Complete Parts 1–6 with the practice handbook, then with a real document collection of at least 20 sections. Share your evaluation table with your instructor in the lesson Q&A.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What should you measure first in a RAG project?
   - A. The font size
   - B. Answer length
   - C. The cost of GPUs
   - D. Retrieval quality, for example recall@k
2. Which documents may you use for the project?
   - A. Any private files you find
   - B. Copyrighted books
   - C. Documents you are allowed to use, without personal data
   - D. Other people's emails
3. What does the 'refusal ok' check test?
   - A. That the assistant refuses exactly when the answer is not in the documents
   - B. Politeness
   - C. Token count
   - D. Speed
4. How should you improve the assistant?
   - A. Change one thing at a time and re-run the evaluation
   - B. Never change it
   - C. Remove the tests
   - D. Change everything at once
5. Which is a real risk of a document assistant?
   - A. Outdated documents give confident but old answers
   - B. Citations are always wrong
   - C. Documents never change
   - D. It cannot read text
6. Why combine embeddings and BM25 in retrieval?
   - A. Keywords catch exact terms, embeddings catch paraphrases
   - B. To double the cost
   - C. It is required by RAG
   - D. BM25 needs embeddings

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.